

■ Lab 4 — Terraform Core Behaviour & Remote State (GitPod)

■ Objectifs du Lab

- Créer votre **bucket S3 personnel** pour le remote state
- Configurer le **backend S3** avec `use_lockfile = true`
- Observer le comportement Terraform face aux modifications **hors Terraform** et **dans le code**

■ Partie A — Bootstrap : Créer votre bucket S3

Reprenez votre environnement GitPod → <https://gitpod.io/workspaces>

```
USERNAME=<votre-prenom>
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
BUCKET_NAME="tf-training-${USERNAME}-${ACCOUNT_ID}"
echo "Bucket : $BUCKET_NAME"
```

```
aws s3api create-bucket \
  --bucket $BUCKET_NAME \
  --region eu-west-1 \
  --create-bucket-configuration LocationConstraint=eu-west-1

aws s3api put-bucket-versioning \
  --bucket $BUCKET_NAME \
  --versioning-configuration Status=Enabled

aws s3api put-public-access-block \
  --bucket $BUCKET_NAME \
  --public-access-block-configuration \
    BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true

echo "Mon bucket S3 : $BUCKET_NAME" >> $HOME/tf-training-info.txt
cat $HOME/tf-training-info.txt
```

■■ Ce bucket sera réutilisé dans ****tous les labs suivants****. Ne le supprimez pas !

■ Partie B — Créer les fichiers

```
cd labs/lab4-state
```

Créez les fichiers dans l'éditeur VS Code GitPod :

``versions.tf``

```
terraform {
  required_version = ">= 1.15.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
provider "aws" { region = "eu-west-1" }
```

``variables.tf``

```
variable "username" {
  description = "Votre prenom"
  type        = string
}
```

``main.tf``

```
resource "aws_security_group" "lab4_sg" {
  name      = "lab4-sg-${var.username}"
  description = "Security group Lab 4 - ${var.username}"
  tags = {
    Name      = "lab4-sg-${var.username}"
    Lab       = "lab4"
    Username  = var.username
  }
}

resource "aws_instance" "lab4_instance" {
  ami          = "ami-0694d931cee176e7d"
  instance_type = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab4_sg.id]
  tags = {
    Name      = "lab4-instance-${var.username}"
    Lab       = "lab4"
    Username  = var.username
  }
}
```

``outputs.tf``

```
output "instance_id" {
  value = aws_instance.lab4_instance.id
}
output "security_group_id" {
  value = aws_security_group.lab4_sg.id
}
```

■ Partie C — Init avec Backend S3

```
BUCKET_NAME=$(grep "bucket" $HOME/tf-training-info.txt | awk '{print $NF}')
USERNAME=<votre-prenom>

terraform init \
  -backend-config="bucket=${BUCKET_NAME}" \
  -backend-config="key=lab4/terraform.tfstate"

terraform apply -var="username=${USERNAME}"

# Vérifier le state dans S3
aws s3 ls s3://${BUCKET_NAME}/lab4/
```

■ Partie D — Modifier hors Terraform

```
INSTANCE_ID=$(terraform output -raw instance_id)
aws ec2 create-tags --resources $INSTANCE_ID --tags Key=ModifiedOutside,Value=true
terraform plan -var="username=${USERNAME}"
terraform refresh -var="username=${USERNAME}"
terraform state show aws_instance.lab4_instance | grep ModifiedOutside
```

■ Partie E — Modifier dans le code

Dans l'éditeur VS Code GitPod, changez `t2.micro` en `t2.small` dans `main.tf` :

```
terraform plan -var="username=${USERNAME}"
```

Remettez `t2.micro` puis vérifiez :

```
terraform plan -var="username=${USERNAME}"
```

Résultat attendu : `No changes`.

■ Partie F — Nettoyage

```
terraform destroy -var="username=${USERNAME}"
```

■■ Ne supprimez ****pas**** votre bucket S3 !

■ Validation du Lab

#	Critère	Validé
1	Bucket S3 créé et nom sauvegardé dans `tf-training-info.txt`	■
2	`terraform init -backend-config` réussi	■
3	State visible dans S3 après `terraform apply`	■
4	`terraform plan` détecte la modification hors Terraform	■
5	`terraform plan` détecte le changement `t2.small`	■
6	`terraform destroy` supprime les ressources AWS	■

■ ****Fin du Lab 4 — Remote state S3 configuré pour toute la formation !****